Digital Design (CS F215) **12-11-2025**

Practice Lab

Lecturer: Harikrishnan N B *Student:*

Question 1 - LFSR (Credits: Anwesh and Arjun)

Part 1: LFSR Design and Simulation

Design and implement a **4-bit Linear Feedback Shift Register (LFSR)** using four positive-edge–triggered D flip-flops. At every clock cycle, the next state is computed as follows:

$$\text{newA}[3] = A[0] \oplus A[2]$$

(where $A[i]$ is the i^{th} flip-flop from the right). After computing the XOR, the register performs a **right shift**, and $A[3]$ is updated with the calculated XOR value. The reset (`set`) is **synchronous active-high** and initializes the register to `1001`. The register state changes only at the **rising edge** of the clock.

Task: Write a Verilog module implementing this logic (behavioral) with:

- Inputs: `clk`, `set`
- Output: `A[3:0]` (register state)

Part 2: LFSR-Based Encryption

Extend your LFSR design to create an **encryption module**. The encryption follows the relation:

$$O_i = K_i \oplus I_i$$

where I is the input bit, K is the key bit generated by the rightmost flip-flop of the LFSR, and O is the output (encrypted bit). After every encryption, the LFSR state shifts as per its feedback rule.

Refer to Table 10.1 for the key generation of the LFSR.

Given:

- Input sequence: 10100111
- Encrypted output: 00010101

Tasks:

1. Derive the key sequence used for encryption.
2. Determine the **initial LFSR state** that generates this key sequence.
3. Identify which flip-flop (among $A[1]$, $A[2]$, or $A[3]$) is XORed with $A[0]$ to form the new $A[3]$.
4. Implement the encryption logic in Verilog (behavioral) with:
   - Inputs: `clk`, `set`, `in`
   - Outputs: `A[3:0]` (new register state), `out` (encrypted bit)

   The state and output must update on the rising edge of the clock.

Table 10.1: Key Generation

State of LFSR	Key Bit Generated
Initial state: 1001	1
1100	0
1110	0
1111	1
0111	1
0011	1
1001	1

Question 2 - K-Maps

Part 1: BCD to Excess-3 Conversion

You are given a 4-bit BCD input $X_3X_2X_1X_0$ representing digits 0–9 (inputs 10–15 are invalid). Design a converter that outputs the corresponding Excess-3 code $Y_3Y_2Y_1Y_0$.

- Use four 4-variable K-maps (one per output bit) to derive minimized Boolean expressions.
- Treat invalid BCD inputs 10–15 as "don't care" conditions.
- Implement the design in Verilog.

Part 2: POS from a 4-Variable Function

Given:

$$F(A, B, C, D) = \Sigma m(1, 4, 5, 8, 10, 12, 15)$$

Obtain a minimized **Product of Sums (POS)** expression.

Part 3: SOP from a 5-Variable Function

Given:

$$F(A, B, C, D, E) = \Sigma m(0, 3, 4, 6, 7, 11, 13, 15, 16, 19, 20, 21, 23, 24, 28, 30)$$

Obtain a minimized **Sum of Products (SOP)** expression.

Question 3 - Combinational Logic

Part 1: BCD Adder

Design a BCD adder that takes two 4-bit BCD digits $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$ (each 0–9) and outputs the BCD sum.

- Implement decimal correction (add 6 when required).
- Outputs: ones digit $S_3S_2S_1S_0$ and carry/tens digit T.

Part 2: 3:8 Decoder from Smaller Decoders

Implement a 3:8 decoder using only 2:4 and 1:2 decoders.

- You may use enables as needed.

Part 3: 8:1 Multiplexer from Smaller Multiplexers

Implement an 8:1 multiplexer using 4:1 and 2:1 multiplexers.

Question 4 - Latches and Flip-Flops

Part 1: SR Latch

Design and implement an SR latch using basic logic gates.

- Inputs: S, R, `Enable`
- Outputs: Q, $\overline{Q}$

Part 2: D Latch using SR Latch

Design and implement a D latch using the SR latch from Part 1.

- Inputs: D, `Enable`
- Outputs: Q, $\overline{Q}$

Part 3: JK Flip-Flop

Design and implement a JK flip-flop using logic gates or a master–slave configuration.

- Inputs: J, K, `clk`, `Enable`
- Outputs: Q, $\overline{Q}$

Question 5 - Registers

4-bit Universal Shift Register

Design and implement a 4-bit universal shift register.

- Inputs: `clk`, `clear`, `MSBin`, `LSBin`, `S1`, `S0`, `D[3:0]` (parallel data in)
- Outputs: `Q[3:0]`
- Operation: On the rising edge of `clk`, perform the function selected by `S1 S0` (see Table 10.2). `clear` is asynchronous active-high and resets `Q` to `0000`.

Table 10.2: Select-Line Operation

S1	S0	Function
0	0	No change
0	1	Shift right ($Q_3 \leftarrow$ `MSBin`, $Q_2 \leftarrow Q_3$, $Q_1 \leftarrow Q_2$, $Q_0 \leftarrow Q_1$)
1	0	Shift left ($Q_0 \leftarrow$ `LSBin`, $Q_1 \leftarrow Q_0$, $Q_2 \leftarrow Q_1$, $Q_3 \leftarrow Q_2$)
1	1	Parallel load ($Q \leftarrow$ `D`)

Question 6 - Counters

Part 1: Mod-6 Counter (Synchronous and Asynchronous)

Design and implement a Mod-6 counter that counts $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 0 \rightarrow \cdots$.

- Provide two implementations:
  1. **Synchronous:** All flip-flops triggered by the same `clk`.
  2. **Asynchronous (Ripple):** Cascaded flip-flops with ripple carry.
- Inputs: `clk`, `clear` (active-high, synchronous or asynchronous as applicable).
- Output: `Q[2:0]`.

Part 2: Mod-7 Counter (Synchronous and Asynchronous)

Design and implement a Mod-7 counter that counts $0 \to 1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 0 \to \cdots$.

- Provide two implementations:
  1. **Synchronous:** All flip-flops triggered by the same `clk`.
  2. **Asynchronous (Ripple):** Cascaded flip-flops with ripple carry.
- Inputs: `clk`, `clear` (active-high, synchronous or asynchronous as applicable).
- Output: `Q[2:0]`.

Part 3: Lab Revisit - Prime Cycle 2.0

Design and implement an asynchronous (ripple) counter that follows the sequence:

$$2 \to 3 \to 5 \to 7 \to 11 \to 13 \to 2 \to 3 \to \cdots$$

- Inputs: `clk`, `clear` (active-high)
- Output: `Q[3:0]` (4 bits to represent $\{2, 3, 5, 7, 11, 13\}$)

Part 4: Tutorial Revisit - Repetition 2.0

Design and implement an asynchronous (ripple) counter that follows the sequence:

$$0 \to 0 \to 1 \to 2 \to 2 \to 3 \to 0 \to 0 \cdots$$